

DSA1101

Introduction to Data Science

Week 12 Tutorial 10

TODAY'S AGENDA

1. Very quick revision of all supervised models
 2. Tutorial Questions Discussion
- No on-site today yippee



HOW WAS UR BREAK?



mine was pretty great

QUICK REVISION

- for each model, we're gonna just focus on how to call it in R, and several pointers that you should not forget during exam
- I'll go a little faster as usual, so at any point if you don't understand, go back to the tutorial notes for that topic to refresh!

LINEAR REGRESSION

```
model <- lm(y ~ x1 + x2, data = my_data)
```

Recall Linear Regression has assumptions, so you may be asked to:

1. check if y is **symmetric** `hist(data$y)`

2. check if **variability of y is stable as x changes**

```
plot(x, y) # if x is numerical  
boxplot(y ~ x) # if x is categorical
```

3. ofc don't forget to confirm that y is **quantitative!**

To make predictions for new data:

```
predict(model, newdata = data)
```

Components you might need to call:

```
summary(model)$coefficients  
summary(model)$r.squared  
summary(model)$adj.r.squared
```

KNN

```
library(class)

model <- knn(train.X, test.X, train.Y, k = 1)
```

knn() function in class package requires:

- All input variables to be **numerical**
- Response variable to be **categorical (factor)**

KNN requires standardization:

```
data = scale(data)
```

To extract winning class probability:

```
model <- knn(train.X, test.X, train.Y, k = 1, prob = TRUE)

attr(model, "prob")
```

To evaluate KNN: Confusion Matrix

```
table(predicted_class, actual_class)
```

or ROC AUC

Confusion Matrix

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Sometimes Negative and Positive will switch places. So please read carefully before answering!

Accuracy =

$$(TP + TN) / (TP + TN + FP + FN)$$

$$TPR (Recall) = TP / (TP + FN)$$

$$FPR = FP / (FP + TN)$$

$$FNR = FN / (TP + FN)$$

$$Precision = TP / (TP + FP)$$

DECISION TREE

```
library(rpart)
library(rpart.plot)

model <- rpart(y ~ x, data = data,
               method = "class",
               control = rpart.control(minsplit = 1, maxdepth = 1, cp = 0.01),
               parms = list(split = "information"))

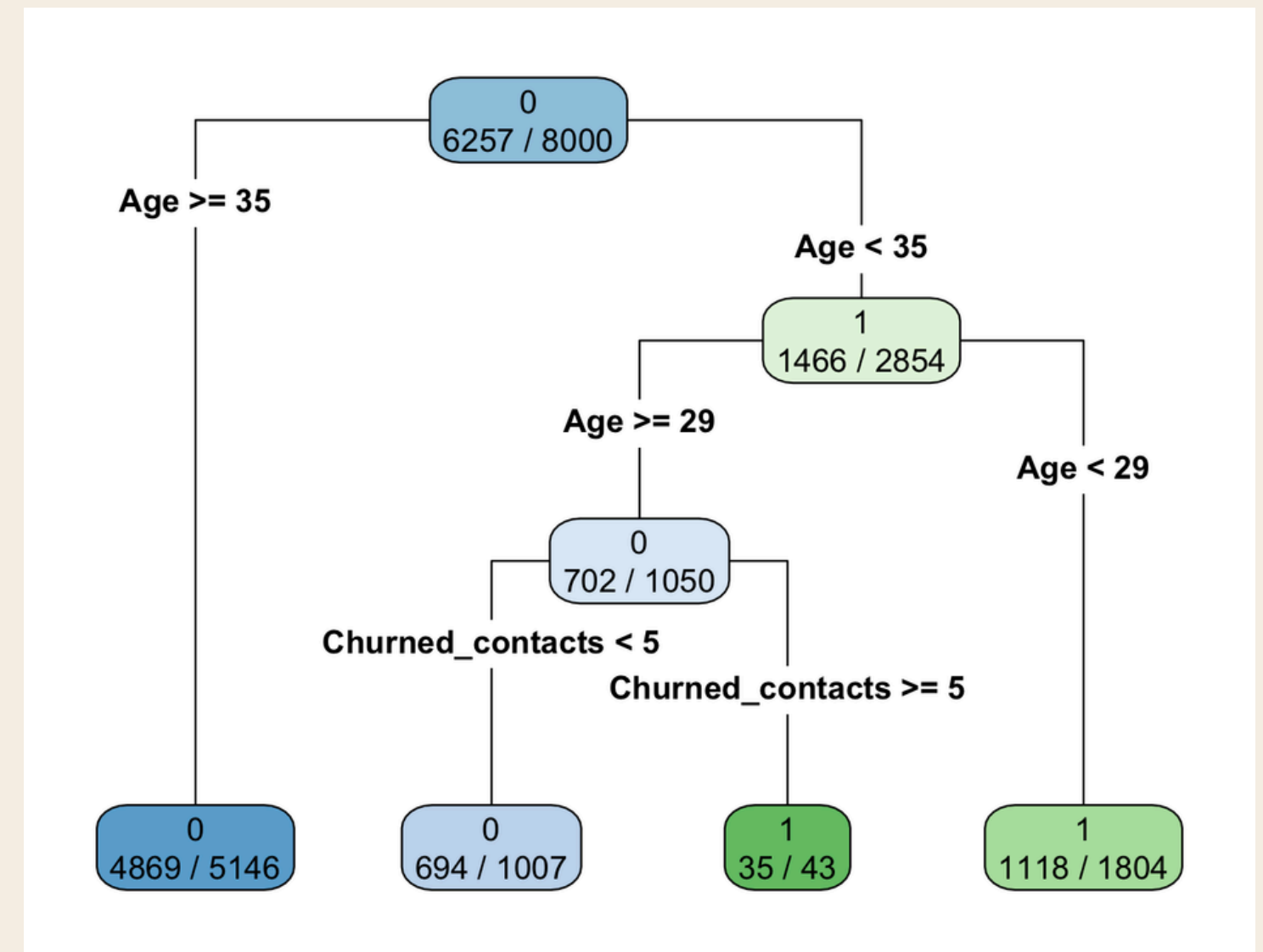
rpart.plot(model, type = 4, extra = 2)
```

To predict new data:

```
# Predicted class
predict(model, newdata, type = "class")

# Class proportions of leaf nodes
predict(model, newdata, type = "prob")
```

- more important features are closer to root
- confusion matrix/ ROC AUC for evaluation



NAIVE BAYES

```
library(e1071)

model = naiveBayes(y ~ x, data)
```

To predict new data:

```
# Predicted class
predict(model, newdata)

# Predicted probabilities
predict(model, newdata, type = "raw")
```

	No	Yes
	0.2060556	0.7939444

- confusion matrix / ROC AUC for evaluation

When extracting $P(Y = 1)$ for ROC, remember to select the “**positive**” column (in this case the second column)

- Same for Decision tree’s (type = “prob”)

	0	1
1	0.3802661	0.61973392
2	0.3802661	0.61973392

DT’s `predict(model, newdata, type = “prob”)`

LOGISTIC REGRESSION

```
model <- glm(y ~ x, data, family = binomial)
```

It's response needs to be 0/1 or "0"/"1"

```
ifelse(data$y == "yes", 1, 0)
```

To predict new data:

```
# Step 1: Predict P(Y = 1)
pred = predict(model, newdata, type = "response")

# Step 2: Classify points using threshold
pred_class = ifelse(pred >= threshold, 1, 0)
```

confusion matrix / ROC AUC for evaluation

Interpreting coefficients (quantitative):

Fixing other variables, a one **<unit>** increase in **<quantitative variable>** increases/decreases the log-odds by **<coef>**

Fixing other variables, a one **<unit>** increase in **<quantitative variable>** multiplies the odds by **<exp(coef)>**

LOGISTIC REGRESSION

```
model <- glm(y ~ x, data, family = binomial)
```

It's response needs to be 0/1 or "0"/"1"

```
ifelse(data$y == "yes", 1, 0)
```

To predict new data:

```
# Step 1: Predict P(Y = 1)
pred = predict(model, newdata, type = "response")

# Step 2: Classify points using threshold
pred_class = ifelse(pred >= threshold, 1, 0)
```

confusion matrix / ROC AUC for evaluation

Interpreting coefficients (categorical):

Fixing other variables, being **<categorical variable = class>** increases/decreases the log-odds by **<coef>**

Fixing other variables, being **<categorical variable = class>** multiplies the odds by **<exp(coef)>**

ROC AUC

```
library(ROCR)

# Step 1: Create ROCR object
rocr_obj <- prediction(positive_class_prob, actual_class)

# Step 2: Compute TPR and FPR
perf <- performance(rocr_obj, measure = "tpr", x.measure = "fpr")

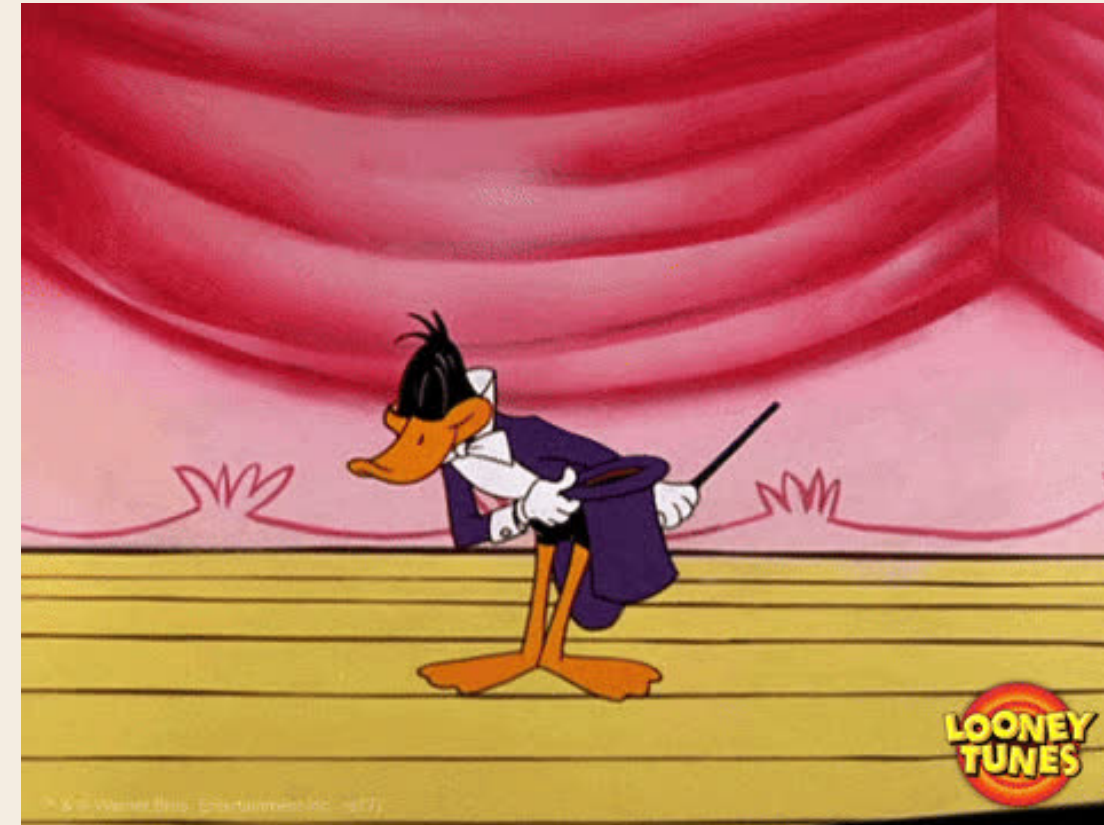
# Step 3: plot ROC curve
plot(perf)

# Step 4: Compute AUC
auc <- performance(rocr_obj, measure = "auc")@y.values[[1]]
```

OFF-SITE QUESTION

Let's go to code!

THANK YOU!



Weekly Slides & Code will be shared on Github (star it for
bookmark):

<https://github.com/kr7001/DSA1101-teaching-materials>